ETS

PEMROGRAMAN OTA BERBASIS BLYNK



Dosen Pengampu:
Dr. Basuki Rahmat, S.Si. MT

Disusun oleh:
Moh Avin Dharma Wijaya (23081010243)

ROBOTIKA A – 081

PROGRAM STUDI INFORMATIKA

FAKULTAS ILMU KOMPUTER

UNIVERSITAS PEMBANGUNAN NASIONAL "VETERAN"

JAWA TIMUR

2026

DAFTAR ISI

BAB I KONFIGURASI DAN KODE PROGRAM 1

1.1 Konsep OTA dan perbedaannya dengan praktikum sebelumnya 3

1.2 Konfigurasi OTA dan datastream Blynk 4

1.3 Kode program 5

BAB II HASIL DAN PEMBAHASAN 16

2.1 Unggah awal melalui kabel dan verifikasi kesiapan OTA 17

2.2 Keadaan dashboard sebelum proses OTA 18

2.3 Pengujian kendali motor pada firmware awal 18

2.4 Pengujian pengaturan kecepatan melalui slider 19

2.5 Pemadaman motor melalui sakelar 21

2.6 Proses unggah melalui jaringan 22

2.7 Verifikasi pasca-OTA pada firmware baru 24

2.8 Pengujian perangkat tanpa kabel data 25

2.9 Rangkuman hasil pengujian 25

2.10 Dokumentasi Praktikum 27

BAB III PENUTUP 28

3.1 Kesimpulan 29

3.2 Saran 30

DAFTAR GAMBAR

Gambar 1. Serial Monitor menampilkan kesiapan OTA dan alamat IP 17
Gambar 2. Dashboard Blynk sebelum OTA dengan firmware v1.0.0 18
Gambar 3. Diagram rangkaian perangkat saat diuji dengan kabel USB tersambung 19
Gambar 4. Kenaikan kecepatan melalui slider menjadi PWM 206 20
Gambar 5. Penurunan kecepatan melalui slider menjadi PWM 195 20
Gambar 6. Keadaan dashboard setelah sakelar dimatikan 21
Gambar 7. Dialog autentikasi saat unggah melalui port jaringan 22
Gambar 8. Dashboard setelah OTA dengan firmware v1.0.1 24
Gambar 9. Diagram rangkaian perangkat saat dioperasikan tanpa kabel data ... 25
Gambar 10. Kit iMCLab terhubung kabel USB saat uji kesiapan OTA 27
Gambar 11. Dashboard Blynk setelah proses unggah melalui jaringan 27

BAB I KONFIGURASI DAN KODE PROGRAM

1.1 Konsep OTA dan perbedaannya dengan praktikum sebelumnya

Tujuan modul keempat ini bukan lagi sekadar menggerakkan motor dari jarak jauh, melainkan memindahkan seluruh jalur distribusi program dari kabel ke udara. Over-The-Air (OTA) adalah mekanisme pengiriman berkas firmware ke board melalui koneksi jaringan WiFi, sehingga citra program yang baru dikirim sebagai paket data, bukan sebagai berkas yang dipindai melalui kabel USB dan port COM seperti pada tiga modul terdahulu. Pada modul ini pengiriman tersebut ditangani pustaka `ArduinoOTA`, yang dijalankan berdampingan dengan `BlynkSimpleEsp32` di dalam satu sketch yang sama.

Keunggulan utama pendekatan ini baru terasa ketika perangkat sudah menyatu dengan tempat pemasangannya. KIT yang tertanam di dalam badan robot, dipaku pada rangka, atau ditempatkan pada ketinggian tertentu tidak lagi perlu dibongkar setiap kali ada perbaikan rutin. Selama perangkat mendapat catu daya dan berada pada jaringan yang sama dengan komputer pengembang, perubahan perilaku motor dapat dikirim tanpa satu pun kabel yang disentuh. Sumber daya pun tidak lagi terikat pada port USB laptop, sehingga adaptor atau powerbank sudah mencukupi.

Kemudahan tersebut membawa konsekuensi yang harus disadari. Bila proses unggah terputus di tengah jalan, board sudah berada pada tahap penulisan partisi cadangan dan berkas firmware yang diterima belum lengkap. ArduinoOTA menyimpan data pada area OTA sebelum diaktifkan, dan restart yang terjadi tepat pada saat transfer belum selesai membuat board menjalankan program lama dengan keadaan periferal yang tidak pasti. Motor bisa saja berhenti mendadak, atau sebaliknya tetap berputar pada duty terakhir ketika catu daya terputus. Karena itu penanganan keadaan darurat menjadi bagian yang tidak terpisahkan dari rancangan program pada modul ini.

Perbedaan menyeluruh dibanding modul sebelumnya dapat dibaca pada tabel berikut.

No	Aspek	Modul sebelumnya (Blynk biasa)	Modul ini (Blynk + OTA)
1	Jalur pengiriman program	Kabel USB melalui port COM	Jaringan WiFi melalui alamat IP board
2	Kedekatan perangkat	Harus terhubung langsung ke laptop	Memadai bila berada pada jaringan yang sama
3	Sumber daya board	Dari port USB laptop	Adaptor atau powerbank
4	Sertifikasi unggah	Tanpa autentikasi	Diperiksa dengan kata sandi OTA
5	Bukti penggantian firmware	Terlihat pada Serial Monitor	Terlihat pada widget versi di dashboard
6	Kendali motor	Berjalan penuh	Tetap berjalan penuh

1.2 Konfigurasi OTA dan datastream Blynk

Identitas perangkat untuk keperluan OTA ditetapkan oleh dua konstanta. `OTA_HOSTNAME` bernilai `imclab-arif`, yaitu nama yang muncul pada daftar port jaringan Arduino IDE sebagai pengganti daftar COM. `OTA_PASSWORD` bernilai `robotika`, dan kata sandi ini diperiksa sebelum tahap penulisan firmware dimulai sehingga komputer lain pada jaringan yang sama tidak bisa serta-merta mengirimkan program ke board.

Untuk menandai keberhasilan penggantian firmware, ditambahkan satu datastream baru berupa V4 yang menampilkan nomor versi. Nilai versi diambil dari makro `FW_VERSION` dan dikirim melalui `Blynk.virtualWrite(V4, FW_VERSION)` di dalam `BLYNK_CONNECTED()`. Dengan cara ini bukti OTA dapat dibaca langsung dari dashboard, tanpa bergantung pada Serial Monitor, dan tetap terlihat meskipun kabel data sudah dicabut. Datastream lain tidak mengalami perubahan dari modul sebelumnya.

Pin virtual	Nama	Tipe	Keterangan
V0	RPM	Double	Hasil pembacaan sensor putaran

V1	PWM	Integer	Nilai duty cycle yang sedang aktif
V2	Slider PWM	Integer	Masukan pengguna untuk mengatur kecepatan
V3	Sakelar ON/OFF	Integer	Tombol penyala dan pemadam motor
V4	FW_VERSION	String	Versi firmware yang berjalan, penanda keberhasilan OTA

Nama template yang dipakai adalah `robot`, dan penetapan tipe data pada masing-masing virtual pin dilakukan pada sisi konsol Blynk, bukan pada program. Nilai `DEFAULT_DUTY` dipakai ketika sakelar dinyalakan dari kondisi mati, sedangkan nilai duty yang sedang berjalan disimpan pada `dutyCycle`.

1.3 Kode program

Berikut adalah isi lengkap `imclab_Blynk_OTA.ino` yang dijalankan pada modul ini. Empat kredensial layanan telah disamarkan, yaitu template ID, token autentikasi Blynk, nama WiFi, dan kata sandi WiFi. Kata sandi OTA dan hostname tetap ditulis apa adanya karena keduanya digunakan sebagai nama perangkat pada daftar port jaringan serta akan diperiksa langsung pada tahap pengujian.

```
/*******************************************************
 * iMCLab + ESP32 + Blynk IoT + OTA
 * RPM Monitoring, Kontrol PWM, dan Update Firmware
 * secara Over-The-Air (ArduinoOTA)
 *******************************************************/

#define BLYNK_TEMPLATE_ID "<TEMPLATE_ID>"
#define BLYNK_TEMPLATE_NAME "robot"
#define BLYNK_AUTH_TOKEN "<AUTH_TOKEN>"

#include <WiFi.h>
#include <BlynkSimpleEsp32.h>
#include <ArduinoOTA.h>

//char ssid[] = "<NAMA_WIFI>";
//char pass[] = "<KATA_SANDI_WIFI>";

//Konfigurasi Wifi
```

```
char auth[] = BLYNK_AUTH_TOKEN;
char ssid[] = "<NAMA_WIFI>";
char pass[] = "<KATA_SANDI_WIFI>";

// ============================================================
// OTA
// ============================================================

// Naikkan nomor ini setiap kali upload, supaya terlihat jelas
// di Serial Monitor bahwa firmware benar-benar sudah berganti.
#define FW_VERSION "1.0.0"

const char* OTA_HOSTNAME = "imclab-arif";
const char* OTA_PASSWORD = "robotika";

// ============================================================
// Motor dan LED
// ============================================================

int ledPin = 2;

int motor1Pin1 = 27;
int motor1Pin2 = 26;
int enable1Pin = 12;

// ============================================================
// Sensor RPM
// ============================================================

const byte pin_rpm = 13;
const int HOLES = 2;

volatile unsigned int rev = 0;
volatile unsigned long lastPulseTime = 0;

const unsigned long DEBOUNCE_US = 500;

// ============================================================
// PWM
// ============================================================

const int freq = 30000;
const int pwmChannel = 0;
const int resolution = 8;

int dutyCycle = 200;

// Nilai yang dipakai saat Switch Blynk dinyalakan dari kondisi mati.
// Angka inilah yang diubah untuk membuktikan OTA lewat dashboard.
const int DEFAULT_DUTY = 200;
```

```
const int DUTY_STEP = 10;
const int DUTY_MIN = 0;
const int DUTY_MAX = 255;

bool motorEnable = false;

// ============================================================
// RPM
// ============================================================

float rpm = 0.0;
unsigned long old_time = 0;

// ============================================================

String inputString = "";

BlynkTimer timer;

// ============================================================
// ISR Encoder
// ============================================================

void IRAM_ATTR isr()
{
  unsigned long now = micros();

  if (now - lastPulseTime > DEBOUNCE_US)
  {
    rev++;
    lastPulseTime = now;
  }
}

// ============================================================
// Sync setelah reconnect Blynk
// ============================================================

BLYNK_CONNECTED()
{
  Blynk.syncVirtual(V2);
  Blynk.syncVirtual(V3);

  // Kirim versi firmware ke widget Label (V4).
  // Inilah bukti OTA yang terlihat langsung di dashboard Blynk.
  Blynk.virtualWrite(V4, FW_VERSION);
}

// ============================================================
// Hitung RPM
// ============================================================
```

```
void hitungRPM()
{
  unsigned long cur_time = millis();

  if (cur_time - old_time >= 1000)
  {
    detachInterrupt(digitalPinToInterrupt(pin_rpm));

    float rot_per_sec = (float)rev / (float)HOLES;

    rpm = rot_per_sec * 60.0;

    rev = 0;

    attachInterrupt(
      digitalPinToInterrupt(pin_rpm),
      isr,
      FALLING);

    old_time = millis();
  }
}

// ============================================================
// Terapkan PWM ke Motor
// ============================================================

void applyMotor()
{
  digitalWrite(motor1Pin1, HIGH);
  digitalWrite(motor1Pin2, LOW);

  ledcWrite(pwmChannel, dutyCycle);

  digitalWrite(ledPin, dutyCycle > 0);

  // Sinkronisasi ke Blynk
  Blynk.virtualWrite(V1, dutyCycle);
  Blynk.virtualWrite(V2, dutyCycle);
  Blynk.virtualWrite(V3, dutyCycle > 0);
}

// ============================================================
// Matikan motor secara paksa (dipakai sebelum OTA)
// ============================================================

void motorDarurat()
{
  dutyCycle = 0;
  motorEnable = false;
```

```
  ledcWrite(pwmChannel, 0);

  digitalWrite(motor1Pin1, LOW);
  digitalWrite(motor1Pin2, LOW);
  digitalWrite(ledPin, LOW);
}

// ============================================================
// Status Serial
// ============================================================

void tampilkanStatus(const char* aksi)
{
  Serial.print(aksi);
  Serial.print(" | PWM=");
  Serial.print(dutyCycle);
  Serial.print(" | RPM=");
  Serial.println(rpm, 1);
}

// ============================================================
// Kirim Data ke Blynk
// ============================================================

void kirimKeBlynk()
{
  Blynk.virtualWrite(V0, rpm);
  Blynk.virtualWrite(V1, dutyCycle);
  Blynk.virtualWrite(V2, dutyCycle);
  Blynk.virtualWrite(V3, dutyCycle > 0);
}

// ============================================================
// Slider PWM dari Blynk (V2)
// ============================================================

BLYNK_WRITE(V2)
{
  dutyCycle = constrain(
    param.asInt(),
    DUTY_MIN,
    DUTY_MAX);

  motorEnable = (dutyCycle > 0);

  applyMotor();

  Serial.print("PWM dari Blynk = ");
  Serial.println(dutyCycle);
}
```

```
// ============================================================
// Tombol ON/OFF dari Blynk (V3)
// ============================================================

BLYNK_WRITE(V3)
{
  motorEnable = param.asInt();

  if (motorEnable)
  {
    if (dutyCycle == 0)
      dutyCycle = DEFAULT_DUTY;
  }
  else
  {
    dutyCycle = 0;
  }

  applyMotor();

  Serial.print("Motor ");
  Serial.println(motorEnable ? "ON" : "OFF");
}

// ============================================================
// Konfigurasi OTA
// ============================================================

void setupOTA()
{
  ArduinoOTA.setHostname(OTA_HOSTNAME);
  ArduinoOTA.setPassword(OTA_PASSWORD);

  ArduinoOTA.onStart([]()
  {
    // Motor WAJIB dimatikan dulu. Board akan restart di tengah
    // proses, jadi motor tidak boleh dibiarkan berputar.
    motorDarurat();

    Serial.println();
    Serial.println("================================");
    Serial.println("OTA dimulai, motor dimatikan");
    Serial.println("================================");
  });

  ArduinoOTA.onProgress([](unsigned int progress, unsigned int total)
  {
    Serial.printf("OTA: %u%%\r", (progress * 100) / total);
  });

  ArduinoOTA.onEnd([]()
  {
    Serial.println();
```

```
    Serial.println("OTA selesai. Board restart...");
  });

  ArduinoOTA.onError([](ota_error_t error)
  {
    Serial.printf("OTA Error [%u]: ", error);

    if (error == OTA_AUTH_ERROR)         Serial.println("Auth gagal");
    else if (error == OTA_BEGIN_ERROR)   Serial.println("Begin gagal");
    else if (error == OTA_CONNECT_ERROR) Serial.println("Connect gagal");
    else if (error == OTA_RECEIVE_ERROR) Serial.println("Receive gagal");
    else if (error == OTA_END_ERROR)     Serial.println("End gagal");
  });

  ArduinoOTA.begin();

  Serial.println("--------------------------------");
  Serial.print("OTA siap. Hostname : ");
  Serial.println(OTA_HOSTNAME);
  Serial.print("Alamat IP          : ");
  Serial.println(WiFi.localIP());
  Serial.println("--------------------------------");
}

// ============================================================
// Setup
// ============================================================

void setup()
{
  Serial.begin(115200);

  pinMode(ledPin, OUTPUT);

  pinMode(motor1Pin1, OUTPUT);
  pinMode(motor1Pin2, OUTPUT);
  pinMode(enable1Pin, OUTPUT);

  pinMode(pin_rpm, INPUT_PULLUP);

  attachInterrupt(
    digitalPinToInterrupt(pin_rpm),
    isr,
    FALLING);

  ledcSetup(
    pwmChannel,
    freq,
    resolution);

  ledcAttachPin(
    enable1Pin,
```

```
    pwmChannel);

  digitalWrite(motor1Pin1, HIGH);
  digitalWrite(motor1Pin2, LOW);

  ledcWrite(pwmChannel, 0);

  old_time = millis();

  // Koneksi ke Blynk (sekaligus menyambungkan WiFi)
  Blynk.begin(auth, ssid, pass);

  // OTA dipasang SETELAH WiFi tersambung
  setupOTA();

  timer.setInterval(
    1000L,
    kirimKeBlynk);

  Serial.println("================================");
  Serial.print("iMCLab + Blynk + OTA Ready  v");
  Serial.println(FW_VERSION);
  Serial.println("================================");
  Serial.println("1 = ON");
  Serial.println("0 = OFF");
  Serial.println("+ = PWM naik");
  Serial.println("- = PWM turun");
  Serial.println("PWM:200");
  Serial.println("R = lihat RPM");
  Serial.println("V = lihat versi firmware & IP");
}

// ============================================================
// Loop
// ============================================================

void loop()
{
  Blynk.run();

  // Melayani permintaan upload OTA dari Arduino IDE
  ArduinoOTA.handle();

  timer.run();

  hitungRPM();

  while (Serial.available() > 0)
  {
    char c = Serial.read();
```

```
if (c == '\n' || c == '\r')
{
  inputString.trim();

  if (inputString.startsWith("PWM:"))
  {
    int val = inputString.substring(4).toInt();

    val = constrain(
      val,
      DUTY_MIN,
      DUTY_MAX);

    dutyCycle = val;
    motorEnable = (dutyCycle > 0);

    applyMotor();

    tampilkanStatus("Set PWM");
  }

  else if (inputString.length() > 0)
  {
    char cmd = inputString.charAt(0);

    if (cmd == '1')
    {
      if (dutyCycle == 0)
        dutyCycle = DEFAULT_DUTY;

      motorEnable = true;

      applyMotor();

      tampilkanStatus("Motor ON");
    }

    else if (cmd == '0')
    {
      dutyCycle = 0;
      motorEnable = false;

      applyMotor();

      Serial.println("Motor OFF");
    }

    else if (cmd == '+')
    {
      dutyCycle += DUTY_STEP;
```

```
            if (dutyCycle > DUTY_MAX)
              dutyCycle = DUTY_MAX;

            motorEnable = true;

            applyMotor();

            tampilkanStatus("PWM NAIK");
          }

          else if (cmd == '-')
          {
            dutyCycle -= DUTY_STEP;

            if (dutyCycle < DUTY_MIN)
              dutyCycle = DUTY_MIN;

            motorEnable = (dutyCycle > 0);

            applyMotor();

            tampilkanStatus("PWM TURUN");
          }

          else if (cmd == 'R' || cmd == 'r')
          {
            Serial.print("RPM = ");
            Serial.print(rpm, 1);
            Serial.print(" | PWM = ");
            Serial.println(dutyCycle);
          }

          else if (cmd == 'V' || cmd == 'v')
          {
            Serial.print("Firmware v");
            Serial.print(FW_VERSION);
            Serial.print(" | Hostname: ");
            Serial.print(OTA_HOSTNAME);
            Serial.print(" | IP: ");
            Serial.println(WiFi.localIP());
          }
        }

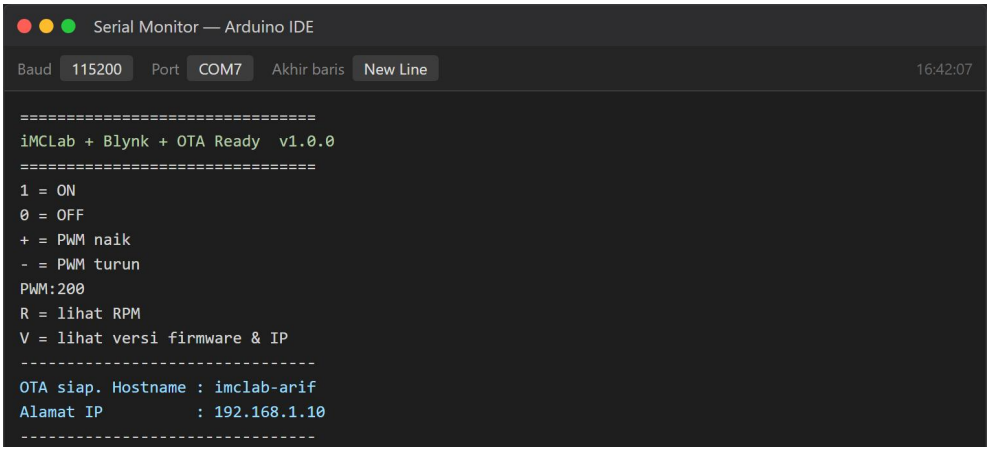
        inputString = "";
      }
      else
      {
        inputString += c;
      }
    }
  }
```

Pemetaan pin yang dipakai tetap sama dengan modul sebelumnya: LED indikator pada GPIO 2 (`ledPin`), kedua pin arah driver motor pada GPIO 27 (`motor1Pin1`) dan GPIO 26 (`motor1Pin2`), pin enable PWM pada GPIO 12 (`enable1Pin`), serta sensor putaran pada GPIO 13 (`pin_rpm`). Parameter PWM tidak berubah, yaitu `freq` 30000 Hz, `pwmChannel` 0, dan `resolution` 8 bit dengan rentang duty 0–255.

BAB II HASIL DAN PEMBAHASAN

2.1 Unggah awal melalui kabel dan verifikasi kesiapan OTA

Firmware pertama pada modul ini tetap ditempuh melalui jalur lama, sebab layanan OTA hanya bisa melayani permintaan bila sketch yang sedang berjalan sudah memuat dan telah memanggil `ArduinoOTA.begin()`. Dengan kata lain, ada satu kali unggah melalui kabel yang tidak dapat dilewati sebelum jaringan dapat dipakai. Sesudah proses tersebut selesai, Serial Monitor pada kecepatan 115200 baud menampilkan blok informasi kesiapan OTA.



Gambar 1. Serial Monitor menampilkan kesiapan OTA dan alamat IP

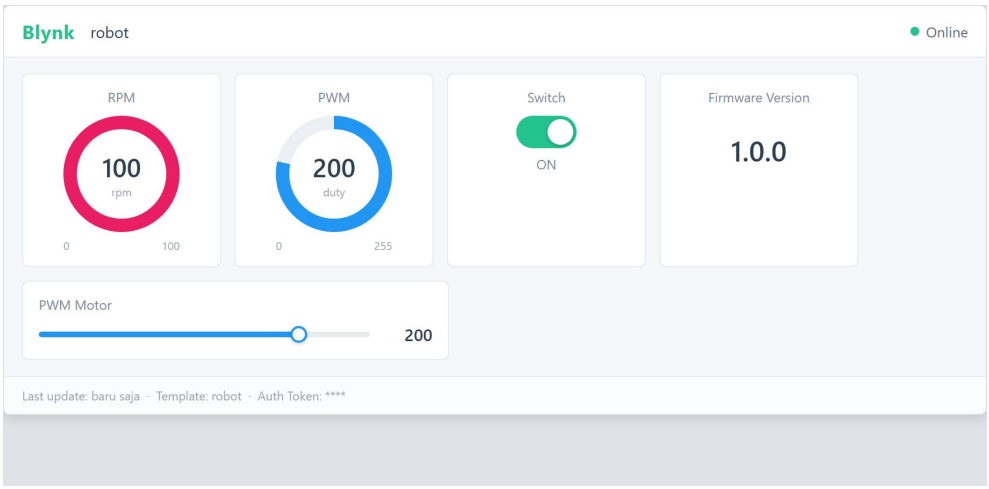
Blok keluaran menyebut hostname `imclab-arif` beserta alamat IP yang diperoleh board dari hotspot, lalu diikuti banner `iMCLab + Blynk + OTA Ready v1.0.0` dan daftar perintah yang sama seperti modul tiga, ditambah satu perintah baru `V` untuk menampilkan versi firmware bersama alamat IP. Alamat IP inilah yang kemudian didaftarkan Arduino IDE sebagai port jaringan, sehingga perangkat dapat ditemukan tanpa perantara kabel.

Satu hal yang perlu dicatat dari urutan keluaran ini adalah bahwa baris OTA dicetak setelah `Blynk.begin(auth, ssid, pass)` selesai. Hal tersebut bukan kebetulan, melainkan konsekuensi dari rancangan `setup()` yang memanggil `setupOTA()`

sesudah koneksi WiFi terbentuk. Bila urutannya dibalik, `ArduinoOTA.begin()` akan berjalan tanpa jaringan dan board tidak akan pernah muncul pada daftar port jaringan.

2.2 Keadaan dashboard sebelum proses OTA

Sebelum percobaan penggantian firmware dimulai, keadaan dashboard Blynk direkam sebagai acuan pembanding. Hal pertama yang diperiksa adalah nilai pada widget versi firmware, karena widget itulah satu-satunya penanda yang tidak ikut berubah ketika motor dijalankan atau dihentikan.

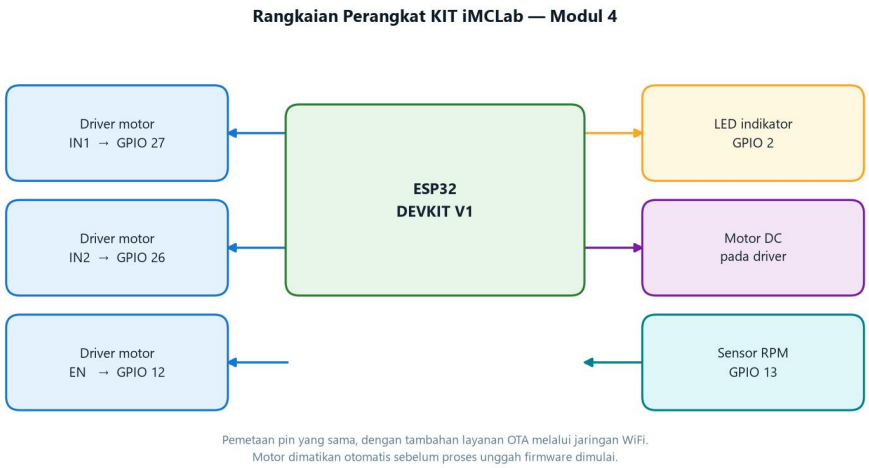


Gambar 2. Dashboard Blynk sebelum OTA dengan firmware v1.0.0

Widget Firmware Version menunjukkan 1.0.0, sesuai nilai makro `FW_VERSION` pada baris `#define FW_VERSION "1.0.0"`. Nilai itu sampai ke dashboard melalui `BLYNK_CONNECTED()`, yang juga mengirim perintah `Blynk.syncVirtual(V2)` dan `Blynk.syncVirtual(V3)` untuk menyelaraskan posisi slider dan sakelar setiap kali perangkat tersambung ulang. Pada gauge PWM dan slider terlihat angka 200, yang berasal dari dua sumber berbeda namun bertemu pada nilai yang sama: `dutyCycle` yang diinisialisasi 200 dan `DEFAULT_DUTY` yang juga bernilai 200. Status perangkat pada konsol Blynk tercatat Online. Rekaman inilah yang akan dibandingkan dengan hasil pada Gambar 8.

2.3 Pengujian kendali motor pada firmware awal

Sebelum firmware diganti, jalur kendali diuji lebih dulu agar bila terjadi penyimpangan setelah OTA, penyebabnya dapat dipersempit menjadi dua kemungkinan saja: pengaruh firmware baru atau pengaruh proses unggah. Yang diamati bukan hanya apakah roda berputar, melainkan apakah seluruh keluaran pada dashboard tetap konsisten dengan keadaan fisik perangkat.



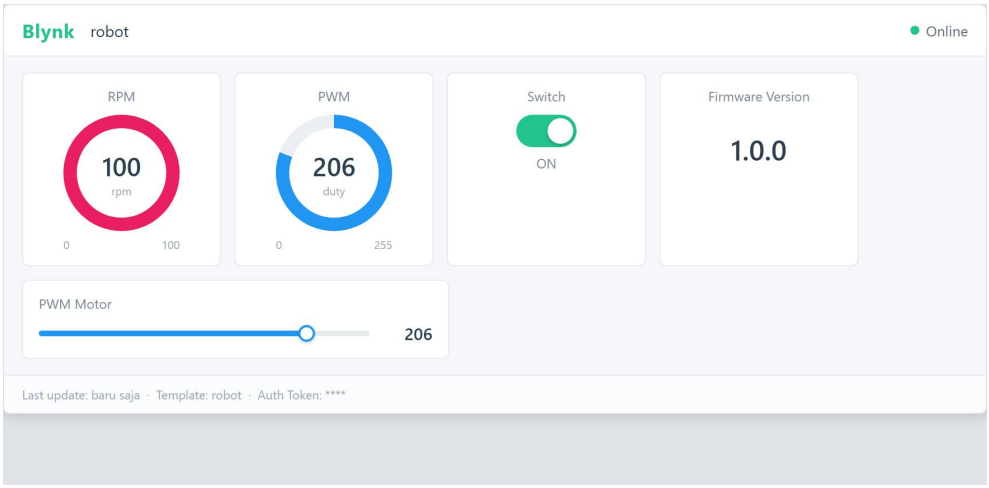
Gambar 3. Diagram rangkaian perangkat saat diuji dengan kabel USB tersambung

Pada langkah ini perangkat masih memperoleh daya dari kabel USB yang tersambung ke laptop. Dashboard menampilkan sakelar dalam keadaan aktif, PWM 200, dan Firmware Version 1.0.0, sementara piringan motor berputar dan LED indikator menyala sebagai tanda `dutyCycle > 0`. Nilai RPM yang terbaca pada dashboard adalah 100. Kondisi inilah yang nantinya dibandingkan dengan Gambar 9 ketika kabel data sudah tidak lagi digunakan.

2.4 Pengujian pengaturan kecepatan melalui slider

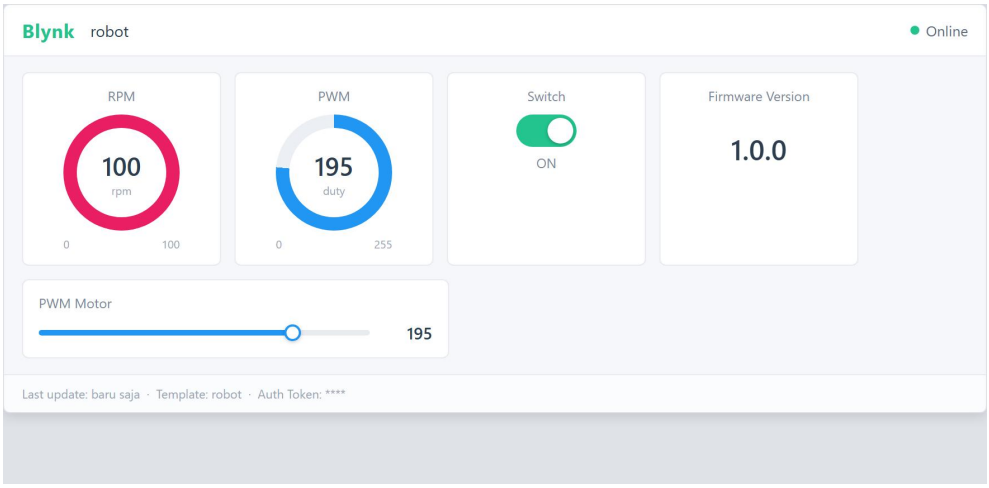
Pengujian berikutnya menyasar datastream V2, yaitu jalur masuk bagi nilai yang dikirim slider dari dashboard. Di sisi program, `BLYNK_WRITE(V2)` menahan nilai tersebut dengan `constrain()` pada rentang `DUTY_MIN` sampai `DUTY_MAX` sebelum

diteruskan, sehingga angka yang berada di luar batas tidak akan merusak keadaan motor.



Gambar 4. Kenaikan kecepatan melalui slider menjadi PWM 206

Gagang slider digeser ke atas dan dashboard mencatat PWM 206. Nilai itu langsung melewati batas 200 yang sebelumnya berlaku karena di bawah kendali slider, `dutyCycle` mengikuti besar kecilnya pergeseran, bukan lagi terpaku pada `DEFAULT_DUTY` yang hanya dipakai saat sakelar dinyalakan dari kondisi mati. Setelah itu slider diturunkan.

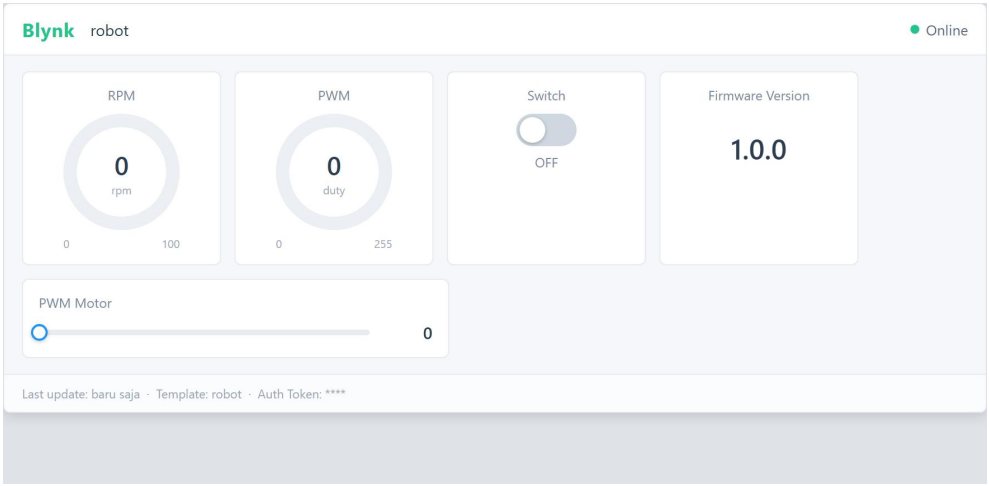


Gambar 5. Penurunan kecepatan melalui slider menjadi PWM 195

Penurunan slider membawa nilai PWM menjadi 195 dan putaran motor ikut melambat. Kedua angka tersebut, 206 dan 195, menunjukkan bahwa `applyMotor()` masih menjalankan tugasnya dengan benar meskipun `loop()` kini juga memanggil `ArduinoOTA.handle()`. Ini penting, sebab bila penanganan OTA menyita waktu proses berlebihan, tanggapan terhadap perubahan slider akan terasa tertunda atau bahkan terlewat.

2.5 Pemadaman motor melalui sakelar

Sakelar pada datastream V3 diuji untuk memastikan bahwa seluruh keluaran dapat dikembalikan ke titik nol dalam satu tindakan. Di dalam `BLYNK_WRITE(V3)`, keadaan sakelar disalin ke `motorEnable`; apabila dinyalakan ketika `dutyCycle` bernilai 0, nilai tersebut diisi ulang dari `DEFAULT_DUTY`, dan apabila dimatikan, `dutyCycle` langsung ditetapkan 0.



Gambar 6. Keadaan dashboard setelah sakelar dimatikan

Setelah sakelar dimatikan, RPM, PWM, dan slider seluruhnya menunjukkan 0, dan LED indikator padam karena `applyMotor()` menulis `digitalWrite(ledPin, dutyCycle > 0)` yang bernilai rendah. Yang tetap bertahan justru widget Firmware Version dengan angka 1.0.0. Kenyataan itu justru memperkuat kedudukan V4 sebagai

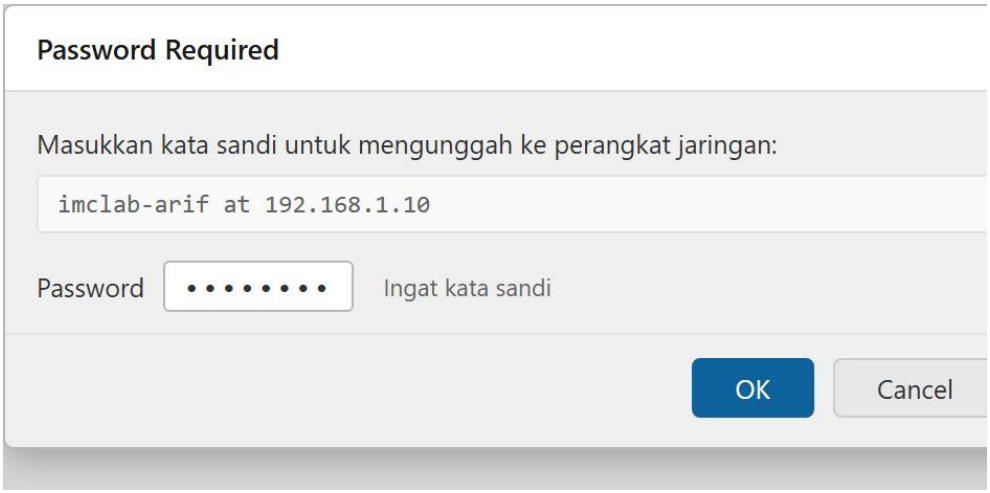
penanda: nilainya tidak bergantung pada apapun selain program yang sedang berjalan, sehingga perubahannya hanya dapat terjadi bila firmware benar-benar diganti.

2.6 Proses unggah melalui jaringan

Tujuan tahap ini adalah membuktikan bahwa citra program dapat berpindah tanpa kabel. Dua baris pada kode disiapkan untuk diubah, yaitu nomor versi dari 1.0.0 menjadi 1.0.1 dan `DEFAULT_DUTY` dari 200 menjadi nilai lain yang mudah dikenali pada dashboard. Nilai versi dipilih karena tampil sebagai teks pada V4, sedangkan `DEFAULT_DUTY` dipilih karena memengaruhi perilaku motor secara langsung dan dapat dibedakan hanya dengan melihat.

Karena firmware yang berjalan saat itu bernilai 1.0.0, sedangkan modul tiga memasang `DEFAULT_DUTY` secara langsung di dalam dua tempat pada `BLYNK_WRITE(V3)` dan pada penanganan perintah `1`, modul ini memperkenalkan konstanta tersendiri bernilai 200 sebagai satu-satunya rujukan. Konstanta itulah yang kemudian diubah pada tahap ini.

Pada Arduino IDE, port yang semula berupa COM diganti dengan port jaringan yang memuat nama hostname perangkat. Sebelum transfer dimulai, IDE meminta kata sandi.



Gambar 7. Dialog autentikasi saat unggah melalui port jaringan

Dialog autentikasi tersebut tidak pernah muncul pada unggah melalui kabel, sehingga kemunculannya sendiri sudah menjadi tanda bahwa pengiriman sedang berlangsung melalui jaringan. Kata sandi yang dimasukkan adalah `robotika`, sama dengan nilai `OTA_PASSWORD`. Di balik layar, `ArduinoOTA.setPassword()` membuat board menolak setiap permintaan yang tidak lolos pemeriksaan sebelum satu byte pun firmware dituliskan. Bila kata sandi salah, jalur `onError` akan mencetak pesan kegagalan autentikasi dan proses dihentikan pada tahap awal.

Pada pelaksanaannya, percobaan pertama sempat berhenti di tengah jalan meskipun tahap pemeriksaan kata sandi sudah terlewati. Sebabnya adalah mekanisme OTA yang mengharuskan board membuka koneksi balik ke komputer pengirim, sementara koneksi masuk tersebut tertahan oleh penyaring jaringan pada laptop. Setelah pengaturan jaringan disesuaikan, transfer berjalan sampai selesai, `onProgress` melaporkan persentase yang terus naik, dan `onEnd` mencetak keterangan bahwa board akan restart.

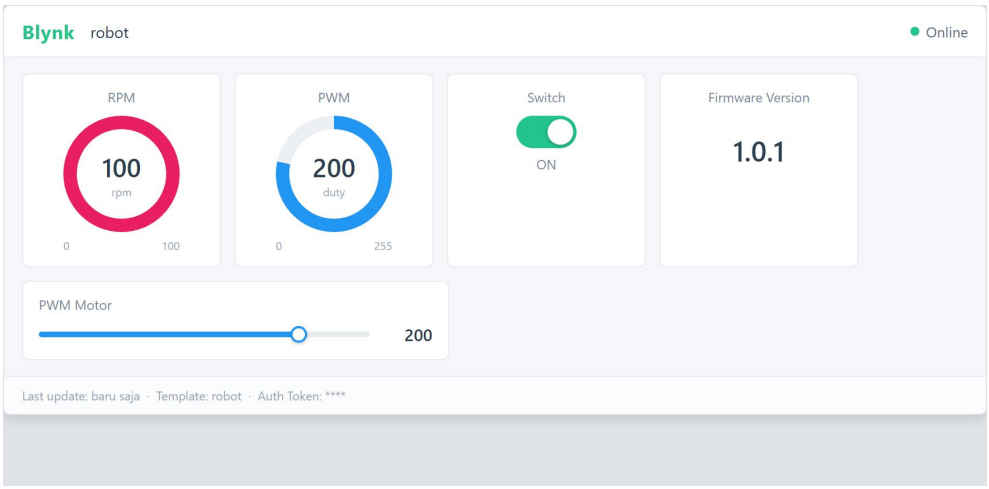
Satu perilaku yang menjadi perhatian tersendiri terjadi tepat ketika proses dimulai. Callback `ArduinoOTA.onStart()` memanggil `motorDarurat()` sebelum transfer berjalan. Fungsi tersebut tidak hanya menetapkan `dutyCycle = 0` dan `ledcWrite(pwmChannel, 0)`, tetapi juga menurunkan `motor1Pin1` dan `motor1Pin2` menjadi LOW dan memadamkan LED. Alasannya bersifat mekanis: board akan restart setelah penulisan selesai, dan selama celah itu keadaan pin tidak lagi diatur oleh program. Membiarkan motor berputar pada kondisi tersebut berarti membiarkan arus mengalir ke driver tanpa kendali, sehingga roda dapat berhenti mendadak atau menarik arus berlebih saat pasokan terputus.

Di sinilah letak perbedaan `motorDarurat()` dari `applyMotor()`. Pada operasi normal, `applyMotor()` selalu menulis `motor1Pin1` HIGH dan `motor1Pin2` LOW untuk menetapkan arah putaran, lalu mengatur besarannya melalui PWM. Pada kondisi darurat, kedua pin arah dibuat LOW. Akibatnya tidak ada beda potensial yang

terbentuk pada kedua terminal motor, sehingga motor benar-benar berhenti dan tidak sekadar berputar dengan duty kecil. Langkah ini melengkapi pemutusan PWM dengan pemutusan arah sekaligus.

2.7 Verifikasi pasca-OTA pada firmware baru

Hakikat pengujian pada tahap ini adalah mencari bukti bahwa program yang berjalan di dalam board benar-benar bukan program yang lama. Karena itu pengamatan diarahkan pada dua hal yang hanya bisa berubah bila baris kode ikut berubah.



Gambar 8. Dashboard setelah OTA dengan firmware v1.0.1

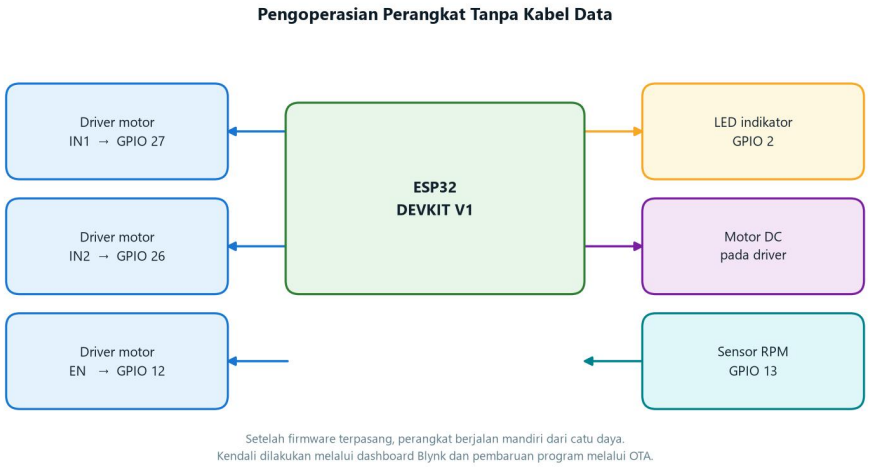
Usai restart otomatis, board menyambung kembali ke server Blynk dan `BLYNK_CONNECTED()` mengirimkan versi baru ke V4. Widget Firmware Version berubah dari 1.0.0 menjadi 1.0.1. Perubahan kedua terlihat ketika sakelar dinyalakan dari kondisi mati: nilai PWM yang muncul adalah `DEFAULT_DUTY` yang baru, bukan 200 seperti sebelumnya. Kedua perubahan itu persis mengikuti dua baris yang disunting, dan tidak satu pun kabel maupun tombol pada board tersentuh selama proses berlangsung.

Perlu ditegaskan bahwa perubahan nomor versi bukan sekadar hiasan. `FW_VERSION` didefinisikan sebagai `"1.0.0"` pada firmware awal dan hanya dikirim ke V4 melalui `BLYNK_CONNECTED()`. Karena itu, selama board masih menjalankan program lama,

label pada dashboard akan tetap menampilkan 1.0.0 berapa kali pun perangkat dinyalakan ulang. Angka 1.0.1 hanya dapat muncul apabila sketch yang baru benar-benar tertanam dan dijalankan.

2.8 Pengujian perangkat tanpa kabel data

Tahap terakhir adalah menguji klaim utama modul ini, yaitu bahwa beban perangkat lunak dapat berubah tanpa kehadiran laptop di dekat perangkat keras. Untuk itu sumber daya dipindahkan ke adaptor dan kabel USB yang sebelumnya terlihat pada Gambar 3 dilepas seluruhnya.



Gambar 9. Diagram rangkaian perangkat saat dioperasikan tanpa kabel data

Perangkat tetap tampil Online pada dashboard dengan Firmware Version 1.0.1. Nilai RPM 100 dan duty awal yang baru pada gauge PWM tetap terlihat, piringan motor tetap berputar, dan LED indikator menyala seperti sebelumnya. Peran laptop pada tahap ini murni sebagai pengirim program dan penampil dashboard, bukan lagi sebagai pemasok daya maupun jalur data. Dengan begitu, rantai pembaruan firmware melalui udara terbukti utuh dari hulu ke hilir.

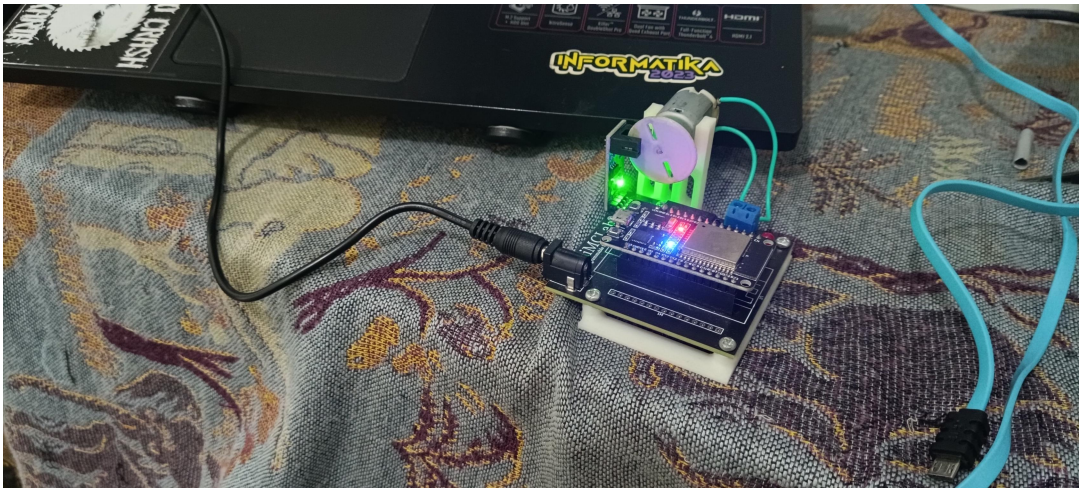
2.9 Rangkuman hasil pengujian

No	Tahap	Hasil pengamatan
1	Unggah awal melalui kabel USB	Serial Monitor menampilkan hostname imclab-arif, alamat IP board, dan banner kesiapan OTA v1.0.0
2	Keadaan sebelum OTA	Firmware Version 1.0.0, gauge PWM 200, slider 200, perangkat Online
3	Slider dinaikkan	PWM 206, putaran motor bertambah cepat
4	Slider diturunkan	PWM 195, putaran motor melambat
5	Sakelar dimatikan	RPM, PWM, dan slider menjadi 0; versi firmware tetap 1.0.0
6	Unggah melalui jaringan	Dialog kata sandi muncul, port berupa hostname perangkat, proses OTA dimulai dengan motor dimatikan
7	Sesudah OTA	Firmware Version 1.0.1, nilai PWM awal berubah mengikuti DEFAULT_DUTY yang baru
8	Tanpa kabel data	Perangkat tetap Online, RPM 100, duty awal yang baru, dan motor tetap dapat dikendalikan

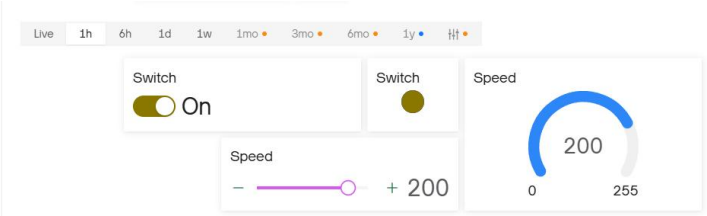
Seluruh tahap berjalan sesuai rancangan. Setiap perubahan yang tampak pada dashboard selalu dapat ditelusuri kembali ke baris program yang menyebabkannya, dan penambahan layanan OTA tidak mengganggu jalur kendali yang sudah berjalan sebelumnya.

2.10 Dokumentasi Praktikum

Praktikum modul keempat menambahkan pembaruan firmware melalui jaringan di atas kendali Blynk. Kit iMCLab diunggah sekali melalui kabel, lalu firmware berikutnya dikirim tanpa kabel data. Foto berikut mendokumentasikan keadaan kit dan dashboard selama proses pembaruan dilakukan.



Gambar 10. Kit iMCLab terhubung kabel USB saat uji kesiapan OTA



Gambar 11. Dashboard Blynk setelah proses unggah melalui jaringan

BAB III PENUTUP

3.1 Kesimpulan

Dari rangkaian pengujian pada modul ini dapat ditarik beberapa kesimpulan berikut.

1. Pustaka `ArduinoOTA` dapat dijalankan berdampingan dengan Blynk dalam satu sketch karena keduanya bekerja di atas ESP32 pada mode station yang sama, sehingga satu koneksi jaringan melayani dua keperluan sekaligus.

2. Unggah pertama melalui kabel USB tetap tidak terhindarkan, sebab perangkat baru mampu melayani permintaan OTA setelah program yang memuat `ArduinoOTA.begin()` berjalan di dalamnya.

3. Keberadaan `setupOTA()` yang dipanggil setelah `Blynk.begin()` membuktikan bahwa urutan penyalaan berpengaruh; tanpa jaringan yang sudah aktif, layanan OTA tidak akan pernah siap menerima permintaan.

4. Pemanggilan `ArduinoOTA.handle()`, `Blynk.run()`, dan `timer.run()` pada setiap iterasi `loop()` berbagi waktu proses yang sama. Penambahan satu pemanggilan menambah beban tiap siklus, namun berdasarkan hasil pengujian, kendali motor melalui slider dan sakelar tetap tanggap.

5. `motorDarurat()` yang dipanggil dari `ArduinoOTA.onStart()` merupakan bentuk penanganan keadaan tidak normal yang penting, karena board akan restart di tengah proses dan kedua pin arah motor dibuat LOW agar tidak ada beda potensial selama catu daya motor beralih.

6. Perbedaan `DEFAULT_DUTY` yang kini menjadi konstanta tersendiri memudahkan pembuktian keberhasilan OTA, sebab satu angka pada satu tempat menghubungkan antara baris kode, perilaku motor, dan tampilan dashboard.

7. Keberhasilan OTA terbukti dari perubahan versi 1.0.0 menjadi 1.0.1 pada widget V4, dan diperkuat oleh perbedaan nilai PWM awal ketika sakelar dinyalakan.

3.2 Saran

Beberapa hal yang dapat dipertimbangkan untuk pengembangan selanjutnya.

1. Nomor versi sebaiknya dinaikkan secara berurutan dan disertai catatan perubahan, agar setiap riwayat firmware dapat ditelusuri tanpa harus membuka kembali sketch lama.

2. Cara masuk perangkat sebaiknya tidak lagi dituliskan langsung di dalam berkas program, melainkan dibaca dari berkas konfigurasi terpisah atau disimpan pada memori non-volatil, sehingga kode dapat dibagikan tanpa perlu disunting ulang.

3. Mekanisme pengaman sebaiknya dilengkapi dengan pemeriksaan nilai checksum atau tanda tangan firmware sebelum board diizinkan restart, agar berkas yang rusak atau datang dari sumber tidak dikenal tidak ikut terpasang.

4. Pengujian ketahanan jaringan perlu ditambahkan, misalnya dengan memutus sambungan tepat pada pertengahan transfer, untuk mengetahui bagaimana perangkat bersikap ketika OTA gagal di tengah jalan.

5. Pemantauan kondisi motor dapat ditingkatkan dengan mengenali pola RPM yang tidak normal, kemudian memanggil `motorDarurat()` secara otomatis sebagai bentuk perlindungan mandiri pada perangkat.